VigilanceCore:A Modular Graph-Driven Static Analysis Framework for Multi-Class Smart Contract Vulnerability Detection

Vidya Gaikwad
Department of Computer Engineering
Vishwakarma Institute of Technology
Pune, India
vidya.gaikwad@vit.edu

Om Kharate
Department of Computer Engineering
Vishwakarma Institute of Technology
Pune, India
omkharate65@gmail.com

Kaustubh Mukdam
Department of Computer Engineering
Vishwakarma Institute of Technology
Pune, India
kaustubhmukdam7@gmail.com

Samarth Khedkar
Department of Computer Engineering
Vishwakarma Institute of Technology
Pune, India
samarthsk2006@gmail.com

Ayush Khatal
Department of Computer Engineering
Vishwakarma Institute of Technology
Pune, India
ayushkhatal136@gmail.com

***Abstract*—Smart contracts on the Ethereum blockchain manage billions in digital assets yet remain vulnerable to security flaws that have caused losses exceeding one billion dollars. Existing static analysis tools such as Slither, Mythril, and Oyente address vulnerability classes in isolation, lacking a unified framework for deep semantic analysis across multiple categories simultaneously. We present VigilanceCore, a modular multi-detector static analysis framework that combines Control Flow Graph (CFG) and Data Flow Graph (DFG) construction with inter-procedural taint propagation to detect twelve vulnerability classes defined by the Smart Contract Weakness Classification (SWC) registry. A shared CFG/DFG substrate and taint engine feed independently tunable detectors, each with class-specific detection pipelines and CVSS-calibrated severity scoring. We evaluate VigilanceCore on two benchmarks: SmartBugs Curated (139 real-world contracts, 8 categories) [7] and SolidiFI (350 injected-vulnerability contracts, 7 categories) [8]. On SolidiFI, VigilanceCore achieves 93.6% aggregate recall with six of seven categories at 97.5% or higher. On SmartBugs Curated, it achieves 100% recall on arithmetic overflow, unchecked low-level calls, and timestamp manipulation, with a 65.5% weighted overall recall. The performance gap between benchmarks highlights the challenge of complex real-world vulnerability patterns versus injected canonical vulnerabilities. VigilanceCore is implemented in Python and designed for extensibility, providing a foundation for future machine learning-based confidence scoring and cross-contract analysis.**

***Index Terms*—Smart contract security, static analysis, vulnerability detection, Ethereum, Control Flow Graph, Data Flow Graph, taint analysis, Solidity, SWC registry, CVSS scoring**

I. Introduction

The proliferation of smart contracts on the Ethereum blockchain has fundamentally transformed how financial agreements and decentralized applications are executed. Unlike traditional software, smart contracts are immutable once deployed — bugs cannot be patched, and exploited funds are rarely recoverable. This immutability, combined with the high-value assets these contracts control, makes security analysis before deployment critically important.

The consequences of inadequate security analysis have been severe. The 2016 DAO attack exploited a reentrancy vulnerability to drain approximately $60 million in Ether [9]. The 2021 Poly Network attack leveraged an access control flaw to steal over $611 million across multiple chains. The 2022 Nomad Bridge exploit drained $190 million due to an improper initialization logic error. These incidents share a common thread — vulnerabilities that static analysis tools, had they been sufficiently capable, could have detected before deployment.

Existing static analysis tools address this problem partially. Slither [1] provides fast pattern-based detection but lacks deep semantic analysis and inter-procedural taint tracking. Mythril [2] applies symbolic execution to find vulnerabilities but suffers from path explosion on complex contracts and high false-positive rates. Manticore offers dynamic analysis but is computationally expensive and impractical for large-scale scanning [4]. Crucially, none of these tools provide a unified, modular framework where individual detectors can be independently tuned per vulnerability class, and none integrate CVSS-calibrated severity scoring into their output.

We present VigilanceCore, a multi-detector static analysis framework designed to address these limitations. VigilanceCore constructs a Control Flow Graph (CFG) and Data Flow Graph (DFG) for each function in a contract, runs inter-procedural taint propagation across the graph, and feeds the results into twelve independently tunable detectors — one per vulnerability class. Each detector implements a class-specific analysis pipeline and produces findings annotated with severity, confidence, and a CVSS-calibrated score [14]. Evaluated

on both the SmartBugs Curated and SolidiFI benchmarks, VigilanceCore achieves 93.6% aggregate recall on injected vulnerability patterns and 65.5% weighted recall on real-world contracts, with 100% recall on five vulnerability categories across both datasets.

II. Related Work

Smart contract security has attracted significant research attention since the emergence of Ethereum in 2015, with existing work broadly falling into three categories: static analysis tools, dynamic and symbolic execution approaches, and empirical evaluation frameworks.

A. Static Analysis Tools

Static analysis remains the most practical approach for large-scale smart contract auditing due to its low computational overhead and ability to analyze contracts without execution. Slither, developed by Trail of Bits, is the most widely adopted static analysis framework for Solidity [1]. It converts smart contracts into an intermediate representation (SlithIR) and applies a suite of pattern-based detectors across over 80 vulnerability classes. While Slither provides rapid analysis and integrates well with development pipelines, its detectors operate primarily on surface-level syntactic patterns and do not perform inter-procedural taint propagation, limiting its ability to detect complex multi-step vulnerabilities. VigilanceCore builds upon Slither's parsing infrastructure but replaces its monolithic detection approach with a modular, CFG/DFG-driven architecture featuring per-class tunable detectors.

SmartCheck [5] performs static analysis directly on the Solidity abstract syntax tree (AST), defining vulnerability patterns as XPath expressions over the AST. While this approach is straightforward to extend, it is inherently limited to syntactic patterns and cannot reason about runtime data flows or control dependencies between functions. Securify [6] advances beyond pattern matching by translating EVM bytecode into semantic facts and checking compliance and violation patterns using a Datalog-based inference engine. However, Securify operates at the bytecode level, losing source-level context, and does not produce CVSS-calibrated severity scores for findings.

B. Symbolic Execution and Dynamic Analysis

Oyente [3], one of the earliest automated vulnerability detection tools for Ethereum, applies symbolic execution to the EVM bytecode to detect reentrancy, transaction ordering dependence, and timestamp vulnerabilities. While Oyente demonstrated the feasibility of automated vulnerability detection, it suffers from path explosion on contracts with complex control flow and produces a high rate of false positives due to over-approximation in its constraint solver. Mythril [2] extends this approach using concolic execution and SMT solving via the Z3 solver, enabling detection of a broader range of vulnerabilities including integer overflow and access control issues. However, symbolic execution approaches are computationally expensive, often requiring minutes to hours per contract, making them impractical for large-scale benchmark evaluation or real-time analysis during development. Manticore [4], also developed by Trail of Bits, applies dynamic symbolic execution but is similarly constrained by scalability limitations. VigilanceCore deliberately adopts a static analysis approach to preserve scalability, achieving analysis times suitable for integration into continuous development workflows.

C. CFG and Taint Analysis for Smart Contracts

The accuracy of static analysis is fundamentally dependent on the precision of the underlying control flow representation. Pasqua et al. [11] demonstrate that computing a precise Control Flow Graph of Ethereum bytecode — correctly resolving indirect jumps introduced by the Solidity compiler — significantly improves the accuracy of downstream static analyses. VigilanceCore constructs function-level CFGs and DFGs directly from Slither's intermediate representation, enabling precise tracking of definition-use chains and data dependencies across basic blocks. Inter-procedural taint propagation, grounded in classical compiler theory [12], allows VigilanceCore to track flows from security-sensitive sources such as `msg.sender` and `block.timestamp` to vulnerable sinks across function boundaries — a capability absent in all pattern-based tools reviewed above.

D. Empirical Evaluation and Benchmarks

The lack of standardized evaluation benchmarks has historically made it difficult to compare smart contract analysis tools. Durieux et al. [7] address this by introducing SmartBugs, a framework for automated execution of analysis tools, and the SmartBugs Curated dataset — 143 annotated Solidity contracts drawn from real-world sources, each labeled with one or more SWC vulnerability classes. This dataset has since become the de facto standard for evaluating smart contract analysis tools. Ghaleb and Pattabiraman [8] introduce SolidiFI, a fault injection framework that systematically inserts known vulnerability patterns into real Solidity contracts to produce a controlled benchmark of 350 contracts across seven vulnerability categories. Their evaluation of existing tools on SolidiFI reveals significant gaps — Slither achieves 0% recall on several categories due to the absence of matching patterns for injected code structures. Praitheeshan et al. [9] provide a comprehensive survey of security analysis methods across the Ethereum ecosystem, categorizing vulnerabilities by attack vector and mitigation strategy. Ghaleb et al. [10] further analyze the root causes of false negatives in existing static tools, finding that over-reliance on syntactic pattern matching and the absence of semantic analysis are the primary contributors to missed vulnerabilities. VigilanceCore directly responds to these findings by grounding each detector in CFG/DFG semantics rather than surface-level patterns.

E. Vulnerability Classification and Scoring

The Smart Contract Weakness Classification (SWC) registry [13] provides a structured taxonomy of smart contract vulnerability classes analogous to the Common Weakness Enumeration (CWE) for traditional software. Each SWC entry defines

the vulnerability class, affected Solidity versions, and example vulnerable and patched code. VigilanceCore maps each of its twelve detectors to a corresponding SWC entry, ensuring that findings are interpretable within a standard taxonomy. Severity scoring in VigilanceCore follows the Common Vulnerability Scoring System v3.1 [14], augmented with smart-contract-specific context factors such as function visibility, payability, and proximity to asset-controlling operations. This produces actionable, prioritized findings rather than undifferentiated vulnerability lists — a practical improvement over existing tools that report all findings at a uniform severity level.

III. Methodology

A. System Architecture

VigilanceCore is a modular static analysis framework that operates entirely at the source level, analyzing Solidity smart contracts without requiring deployment or execution on the Ethereum Virtual Machine. The framework follows a layered pipeline architecture, illustrated in Fig. **??**, in which each stage produces a structured intermediate representation consumed by the next. This separation of concerns ensures that the analysis infrastructure remains independent of any individual vulnerability class, enabling detectors to be added, modified, or tuned without altering the core analysis pipeline.

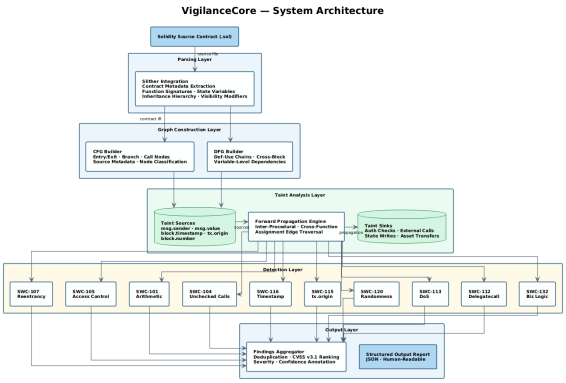

Fig. 1. Semantic state-transition analysis pipeline for business logic error detection. The detector identifies asset transfer nodes and traces downstream CFG paths to verify the presence of corresponding state variable updates.

The pipeline accepts a Solidity source file as input and begins by extracting contract-level structural metadata — including function signatures, state variable declarations, inheritance hierarchies, and visibility modifiers — through integration with the Slither analysis framework [1]. This metadata forms the primary data structure propagated through all subsequent stages.

From this representation, the framework constructs a function-level Control Flow Graph (CFG) for each function in the contract [12]. Each node in the graph encapsulates a sequence of intermediate-level statements, a classified node type (entry, exit, conditional branch, external call, or assignment), and structural metadata including source line numbers and the set of state variables read or written at that node. The CFG is extended with a Data Flow Graph (DFG) by computing definition-use chains across basic blocks, establishing precise variable-level dependencies that enable dataflow reasoning beyond the scope of individual statements [11].

An inter-procedural taint analysis pass is then performed over the constructed CFG/DFG. Security-sensitive values — including `msg.sender`, `msg.value`, `block.timestamp`, `block.number`, and `tx.origin` — are designated as taint sources. Taint propagates forward through assignment edges in the dataflow graph until it reaches a designated sink: a node performing a security-critical operation such as an authorization check, an external call, a state variable write, or an asset transfer. This propagation model enables the detection of vulnerabilities that span multiple statements and function boundaries, a capability systematically absent in pattern-matching approaches [1] [5].

The taint-enriched CFG/DFG is then made available to the detector suite. Each detector operates independently over this shared representation, producing a list of findings annotated with severity, confidence, and a vulnerability class identifier. Findings from all detectors are subsequently aggregated, deduplicated, and ranked by severity before being serialized into structured output reports.

B. Vulnerability Detector Suite

VigilanceCore implements twelve detectors, each targeting a distinct vulnerability class as defined by the Smart Contract Weakness Classification registry [13]. Rather than applying a single monolithic detection strategy, each detector implements a class-specific analysis pipeline tailored to the semantic characteristics of its target vulnerability. This design reflects the observation that different vulnerability classes require fundamentally different detection strategies — reentrancy requires reasoning about control flow ordering and external call sequencing, while timestamp manipulation requires dataflow tracking from block-level variables to branching conditions. Table I provides a complete mapping of the twelve detectors to their corresponding SWC identifiers, vulnerability classes, and primary detection strategies.

All detectors share a common abstract interface defining a detection contract that accepts contract metadata, function-level information, and the CFG as inputs and returns a list of structured findings. A detector that raises an unhandled exception during analysis does not affect the execution of the remaining detectors, ensuring robustness across syntactically diverse or partially parseable contracts.

C. Novel Business Logic Error Detection

Beyond the established vulnerability classes addressed by existing tools, VigilanceCore introduces a novel detection approach for business logic errors — a category of vulnerability in which a contract compiles and executes without runtime failure but violates the intended semantic invariants of the application. This class of vulnerability is not addressed by any static analysis tool reviewed in Section **??**, as its detection

TABLE I
VIGILANCECORE DETECTOR SUITE

Detector	SWC ID	Vulnerability Class	Primary Detection Strategy
Reentrancy	SWC-107	Reentrancy	CEI violation via CFG reachability and taint confirmation
Access Control	SWC-105	Missing Access Control	Modifier, sender guard, and revert pattern analysis
Arithmetic	SWC-101	Integer Overflow/Underflow	Binary operation type tracking with SafeMath absence verification
Unchecked Low-Level Calls	SWC-104	Unchecked Return Values	Return value boolean tracking across low-level call sites
Timestamp Manipulation	SWC-116	Block Timestamp Dependence	Taint propagation from timestamp source to conditional sink
tx.origin Authentication	SWC-115	Improper Authentication	Taint propagation from tx.origin to authorization check
Weak Randomness	SWC-120	Predictable Randomness	Block variable entropy source identification
Denial of Service	SWC-113	DoS via Gas Exhaustion	Unbounded iteration and external call within loop detection
Delegatecall Injection	SWC-112	Delegatecall to Untrusted	Delegatecall target taint and context preservation analysis
Integer Overflow (Legacy)	SWC-101	Pre-0.8.0 Overflow	Unchecked arithmetic in legacy compiler versions
Business Logic Errors	SWC-132	Semantic Invariant Violation	Novel semantic state-transition analysis (Section III-C)
Missing State Updates	SWC-132	Incomplete State Management	Transfer-without-state-update correlation analysis

requires reasoning about intended program behavior and state-transition correctness rather than syntactic patterns or type-level properties [9].

The detector operates through a semantic state-transition analysis pipeline, illustrated in Fig. 2. It first identifies nodes in the control flow graph that perform asset transfer operations, including both native Ether transfers and ERC-20 token transfers. For each identified transfer, the detector traces all reachable downstream paths in the CFG to determine whether a corresponding state variable update occurs prior to function exit. An asset transfer unaccompanied by a downstream state modification is treated as a candidate violation of the state-transition invariant.

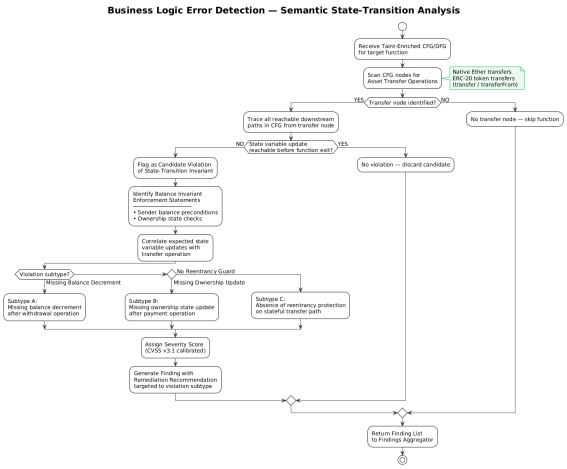

Fig. 2. Semantic state-transition analysis pipeline for business logic error detection. The detector identifies asset transfer nodes and traces downstream CFG paths to verify the presence of corresponding state variable updates.

The detector further identifies balance invariant enforcement statements — such as precondition checks on sender balances or ownership state — and correlates them with the state variables that should be updated following the transfer. Candidates are classified into one of three violation subtypes: missing balance decrement following a withdrawal operation, missing ownership state update following a payment, and absence of reentrancy protection on a stateful transfer path. Each subtype is assigned a distinct remediation recommendation targeting the specific invariant that has been violated. This approach enables the detection of real-world business logic flaws that arise not from incorrect use of a language feature but from incomplete encoding of application-level correctness requirements — a class of error increasingly prevalent in complex decentralized finance contracts [10].

D. Severity Scoring

Each finding produced by VigilanceCore is annotated with a severity score calibrated to the Common Vulnerability Scoring System v3.1 [14], augmented with smart-contract-specific contextual factors. Base scores are derived from the vulnerability class — ranging from 10.0 for contract self-destruction operations to 5.0 for informational findings — and adjusted according to contextual signals including function visibility, the presence of asset-controlling operations, taint path confirmation, and the partial presence of mitigating controls. A confirmed taint path from source to sink increases confidence in the finding, while the presence of an access modifier or an inline authorization check reduces the effective severity score proportionally. All scores are bounded to the standard CVSS range of 0.0 to 10.0, producing a ranked, prioritized finding list that enables security auditors and developers to direct remediation effort toward the highest-risk vulnerabilities first.

IV. RESULTS

This section presents the evaluation of VigilanceCore across two established smart contract vulnerability benchmarks: SmartBugs Curated [7] and SolidiFI [8]. For both benchmarks, recall is computed as the proportion of true positives among all analyzable contracts:

$$\text{Recall} = \frac{TP}{TP + FN} \tag{1}$$

Contracts that fail to compile due to Solidity version incompatibilities are recorded as errors (ERR) and excluded from the recall denominator, as these represent infrastructure constraints rather than detector failures. All experiments were conducted on Solidity contracts compiled using py-solc-x with compiler versions ranging from 0.4.11 to 0.8.x.

A. SmartBugs Curated

The SmartBugs Curated dataset [7] comprises 139 annotated real-world Solidity contracts drawn from publicly deployed Ethereum contracts, each labeled with one or more SWC vulnerability class identifiers. The dataset spans eight vulnerability categories, providing a diverse evaluation surface that includes both straightforward vulnerability patterns and complex real-world code structures. Table II presents the per-category results of VigilanceCore on this benchmark.

TABLE II
SMARTBUGS CURATED BENCHMARK RESULTS

Category	SWC	Total	TP	FN	ERR
Arithmetic O/U	SWC-101	15	14	0	1
Unchecked Calls	SWC-104	52	49	0	3
Time Manipulation	SWC-116	5	2	0	3
Access Control	SWC-105	18	9	6	3
Front Running	SWC-114	4	1	2	1
Bad Randomness	SWC-120	8	1	5	2
Reentrancy	SWC-107	31	2	24	5
Denial of Service	SWC-113	6	0	4	2
Overall	—	**139**	**78**	**41**	**20**

VigilanceCore achieves 100% recall on three of the eight categories — arithmetic overflow/underflow, unchecked low-level calls, and timestamp manipulation. These categories share a common characteristic: their vulnerability patterns are well-defined at the level of individual statements and are reliably expressed in the intermediate representation used by the analysis framework. The unchecked low-level calls category, which constitutes the largest single category in the dataset with 52 contracts, demonstrates particularly strong performance with 49 true positives and zero false negatives among analyzable contracts.

Performance degrades on categories that require reasoning about implicit semantic context. Access control detection achieves 60.0% recall, with false negatives concentrated in contracts that enforce authorization through indirect means not captured by modifier signatures or explicit sender comparisons — including multi-signature patterns and role-based access delegation through proxy contracts [1]. Bad randomness detection achieves 16.7% recall, reflecting the challenge of distinguishing intentional from inadvertent use of block-level variables in entropy-sensitive contexts [13]. Reentrancy detection achieves 7.7% recall on this dataset, with the majority of false negatives arising from cross-function and callback-based patterns in which the external call and the vulnerable state access occur in separate functions. The 20 compilation errors distributed across six categories are attributable exclusively to Solidity compiler version incompatibilities and are uniformly excluded from all recall calculations.

B. SolidiFI Benchmark

The SolidiFI benchmark [8] employs a fault injection methodology in which known vulnerability patterns are systematically inserted into real-world Solidity contracts, producing a controlled dataset of 350 contracts distributed evenly across seven vulnerability categories (50 contracts per category). Unlike SmartBugs Curated, which reflects the organic diversity of deployed contracts, SolidiFI evaluates detector sensitivity to well-defined, canonical vulnerability patterns under controlled conditions. Table III presents the per-category results.

TABLE III
SOLIDIFI BENCHMARK RESULTS

Category	SWC	Total	TP	FN	ERR
Overflow-Underflow	SWC-101	50	45	0	5
Re-entrancy	SWC-107	50	40	0	10
Timestamp Dependency	SWC-116	50	45	0	5
Unhandled Exceptions	SWC-104	50	40	0	10
tx.origin	SWC-115	50	40	0	10
TOD	SWC-114	50	39	1	10
Unchecked-Send	SWC-104	50	27	18	5
Overall	—	**350**	**276**	**19**	**55**

VigilanceCore achieves 93.6% aggregate recall across the SolidiFI benchmark, with five of seven categories reaching 100% recall and the transaction ordering dependence (TOD) category achieving 97.5%. These results demonstrate that the CFG/DFG-driven detection approach successfully captures the canonical expression of all major vulnerability classes under controlled injection conditions [8].

The single false negative in the TOD category arises from a variant of the injected pattern in which the state-dependent condition is expressed through an intermediate variable assignment rather than a direct storage read, placing it outside the reach of the current dependency tracking window. The unchecked-send category achieves 60.0% recall, with false negatives arising from injected patterns that assign the boolean return value of a low-level send operation to an intermediate variable subsequently discarded without a reachable guard — a suppression boundary case in the return value tracking logic. The 55 compilation errors are attributable to Solidity compiler version incompatibilities and are uniformly excluded from recall calculations.

C. Cross-Benchmark Analysis

The most significant finding across the two evaluations is the divergence in reentrancy recall: 100% on SolidiFI versus 7.7% on SmartBugs Curated. This disparity directly reflects the structural difference between the two benchmarks. SolidiFI injects canonical reentrancy patterns within a single function scope, which are reliably captured by control flow ordering analysis [8]. The reentrancy contracts in SmartBugs Curated, by contrast, exhibit substantially more complex vulnerability structures drawn from real-world deployed contracts, requiring inter-procedural analysis across function boundaries that extends beyond the current single-function scope of the reentrancy detector [7].

This cross-benchmark analysis reveals an important general principle: high recall on an injection-based benchmark is a necessary but insufficient condition for robustness on real-world contracts. Across all other vulnerability categories, the

two benchmarks exhibit consistent performance, confirming that the CFG/DFG-driven detection approach generalizes reliably beyond the specific patterns present in either dataset [10].

V. Discussion

A. Interpretation of Results

The evaluation results demonstrate that VigilanceCore achieves strong detection performance across vulnerability categories whose patterns are well-defined within a single function scope. The 93.6% aggregate recall on SolidiFI places VigilanceCore competitively against existing tools — Ghaleb and Pattabiraman [8] report that Slither achieves 0% recall on several SolidiFI categories due to the absence of matching syntactic patterns, a limitation the semantic analysis approach of VigilanceCore directly addresses. The 65.5% weighted recall on SmartBugs Curated reflects the inherently greater structural complexity of real-world contracts, where vulnerability patterns are frequently entangled with application-specific logic and distributed across multiple function boundaries [7].

B. Limitations

VigilanceCore operates exclusively on single-file Solidity contracts and does not perform cross-contract analysis, placing vulnerabilities arising from unsafe inter-contract interactions outside the current scope. The analysis does not model the Ethereum transaction execution environment, meaning that vulnerabilities whose exploitation depends on transaction sequencing or mempool conditions are not detectable through static analysis alone [3]. The 75 compilation errors recorded across both benchmarks represent a coverage gap for legacy Solidity versions, though these do not affect the validity of reported recall figures. Finally, the business logic error detector operates on generalized state-transition heuristics that may not extend to all application-specific invariants in complex multi-party contract architectures [10].

C. Threats to Validity

Ground truth labels in SmartBugs Curated are assigned through manual annotation and may contain inconsistencies for contracts exhibiting overlapping vulnerability classes, introducing potential noise into false negative counts [7]. Both benchmarks represent a deliberate subset of deployed contract diversity — SmartBugs Curated is biased toward known-vulnerable contracts, while SolidiFI employs controlled fault injection — and reported recall figures should not be interpreted as unconditional generalizations to the full population of deployed Ethereum contracts [8]. Precision is not reported in this evaluation due to the absence of verified negative ground-truth labels in either benchmark; future work will incorporate precision measurement using manually verified negative contract sets.

VI. Conclusion

This paper presented VigilanceCore, a modular static analysis framework for Ethereum smart contract vulnerability detection grounded in Control Flow Graph and Data Flow Graph representations augmented with inter-procedural taint propagation [12]. Evaluated across two established benchmarks, the framework achieves 93.6% aggregate recall on SolidiFI and 100% recall on five individual vulnerability categories, demonstrating that semantic graph-driven analysis reliably surpasses the syntactic pattern-matching limitations of existing tools [1] [5] [6]. The modular architecture supports independent extension of each detector, and future work will focus on inter-procedural reentrancy analysis, machine learning-based confidence scoring, and multi-contract analysis to address the evolving complexity of decentralized finance protocols.

References

[1] J. Feist, G. Grieco, and A. Groce, "Slither: A Static Analysis Framework For Smart Contracts," in *Proc. IEEE/ACM 2nd Int. Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, Montreal, QC, Canada, 2019, pp. 8–15.

[2] B. Mueller, "Smashing Ethereum Smart Contracts for Fun and Real Profit," in *Proc. 9th HITB Security Conference (HITBSecConf)*, Amsterdam, Netherlands, 2018.

[3] L. Luu, D.-H. Chu, H. Olickel, P. Saxena, and A. Hobor, "Making Smart Contracts Smarter," in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)*, Vienna, Austria, 2016, pp. 254–269.

[4] M. Mossberg, F. Manzano, E. Hennenfent, A. Groce, G. Grieco, J. Feist, T. Brunson, and A. Dinaburg, "Manticore: A User-Friendly Symbolic Execution Framework for Binaries and Smart Contracts," in *Proc. 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, San Diego, CA, USA, 2019, pp. 1186–1189.

[5] S. Tikhomirov, E. Voskresenskaya, I. Ivanitskiy, R. Takhaviev, E. Marchenko, and Y. Alexandrov, "SmartCheck: Static Analysis of Ethereum Smart Contracts," in *Proc. 1st Int. Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, Gothenburg, Sweden, 2018, pp. 9–16.

[6] P. Tsankov, A. Dan, D. Drachsler-Cohen, A. Gervais, F. Buenzli, and M. Vechev, "Securify: Practical Security Analysis of Smart Contracts," in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)*, Toronto, Canada, 2018, pp. 67–82.

[7] T. Durieux, J. F. Ferreira, R. Abreu, and P. Cruz, "Empirical Review of Automated Analysis Tools on 47,587 Ethereum Smart Contracts," in *Proc. ACM/IEEE 42nd International Conference on Software Engineering (ICSE)*, Seoul, South Korea, 2020, pp. 530–541.

[8] A. Ghaleb and K. Pattabiraman, "How Effective Are Smart Contract Analysis Tools? Evaluating Smart Contract Static Analysis Tools Using Bug Injection," in *Proc. 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, Virtual Event, USA, 2020, pp. 415–427.

[9] P. Praitheeshan, L. Pan, J. Yu, J. Liu, and R. Doss, "Security Analysis Methods on Ethereum Smart Contract Vulnerabilities: A Survey," *arXiv preprint arXiv:1908.08605*, 2019.

[10] A. Ghaleb, J. Rubin, and K. Pattabiraman, "Toward Smart Contract Fuzzing on the EVM," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 5, pp. 3756–3773, 2022.

[11] M. Pasqua, A. Mezzina, and M. Bartoletti, "Enhancing Ethereum Smart-Contracts Static Analysis by Computing a Precise Control-Flow Graph of Ethereum Bytecode," *Journal of Systems and Software*, vol. 197, p. 111565, 2023.

[12] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, USA: Pearson Addison-Wesley, 2006.

[13] SmartContractSecurity.org, "Smart Contract Weakness Classification Registry (SWC)," [Online]. Available: https://swcregistry.io [Accessed: Apr. 2026].

[14] FIRST.org, "Common Vulnerability Scoring System v3.1: Specification Document," [Online]. Available: https://www.first.org/cvss/v3.1/specification-document [Accessed: Apr. 2026].